

AWS Highly Available Web Application

ALB + Auto Scaling + Self-Healing EC2 Infrastructure

1. Project Overview

This project demonstrates the design and implementation of a highly available, scalable, and self-healing web application infrastructure on AWS.

The application is hosted on Amazon EC2 instances distributed across multiple Availability Zones. An **Application Load Balancer (ALB)** receives incoming HTTP traffic and distributes it to healthy EC2 instances through a Target Group.

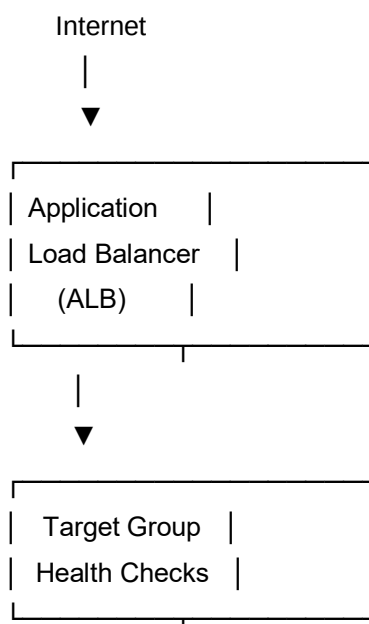
An **Auto Scaling Group (ASG)** manages the EC2 instances and maintains the desired capacity of the application. A Launch Template provides a consistent blueprint for every EC2 instance, including the AMI, instance type, security group, key pair, storage configuration, and User Data automation.

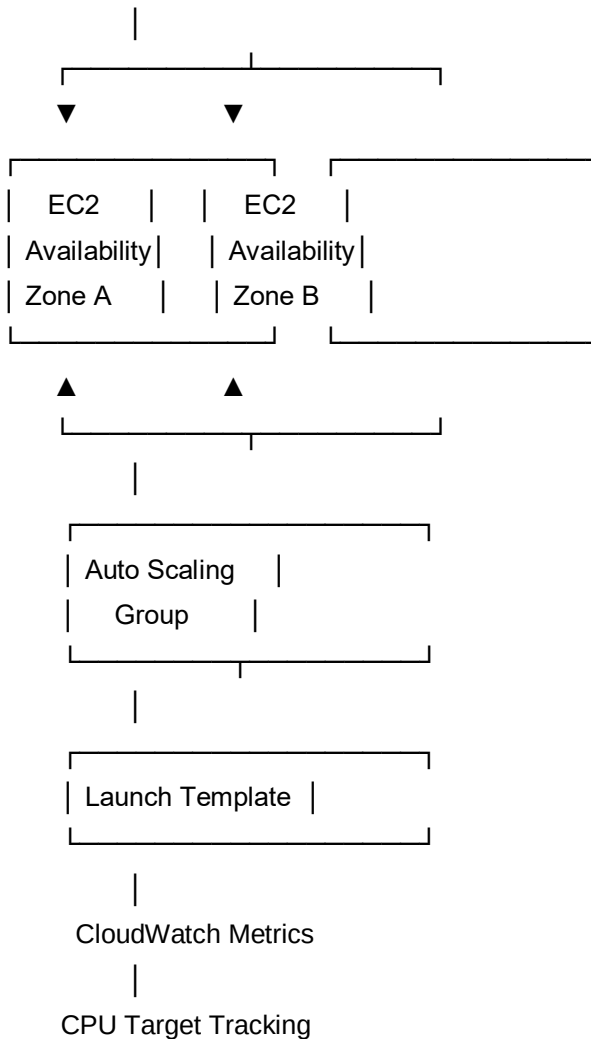
The infrastructure also uses **Target Tracking scaling based on average CPU utilization**. This allows the Auto Scaling Group to automatically adjust the number of EC2 instances according to workload.

A failure scenario was deliberately tested by terminating an active EC2 instance. The Auto Scaling Group detected the capacity reduction, launched a replacement instance, and the new instance was automatically registered with the Target Group and passed its health checks.

The project therefore demonstrates not only deployment, but also **load balancing, health monitoring, automatic scaling, multi-AZ availability, and self-healing behavior**.

Final Architecture





2. Project Objective

The objective of this project was to build a **production-style, highly available web application infrastructure on AWS** and understand how the individual AWS services work together rather than configuring them independently.

The project focused on achieving the following:

- Deploy EC2 instances across **multiple Availability Zones**.
- Use an **Application Load Balancer** to distribute incoming HTTP traffic.
- Use a **Target Group** to maintain backend instances and perform health checks.
- Use a **Launch Template** as a reusable EC2 configuration blueprint.
- Use **User Data** to automatically install and configure the Apache web server.
- Use an **Auto Scaling Group** to maintain the required number of EC2 instances.
- Configure **Target Tracking Scaling** based on average CPU utilization.
- Restrict direct access to EC2 instances using **Security Group references**.
- Verify application availability through the ALB DNS endpoint.
- Simulate an instance failure by terminating an active EC2 instance.
- Verify that the Auto Scaling Group automatically launches a replacement instance.
- Verify that the replacement instance becomes healthy and is automatically registered with the Target Group.
- Understand the relationship between **load balancing, health checks, scaling, and self-healing**.

The final goal was not simply to deploy the infrastructure, but to **observe and verify its behavior under normal operation and failure conditions**.

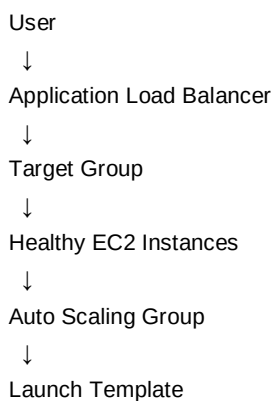
3. AWS Services Used

The project combines multiple AWS services, with each service handling a specific responsibility in the architecture.

AWS Service	Role in the Project
Amazon EC2	Provides the compute instances running the Apache web server and application webpage.
Amazon EC2 Launch Template	Defines the reusable configuration used to launch identical EC2 instances.
Amazon VPC	Provides the networking environment in which the ALB and EC2 instances operate.
Security Groups	Control inbound traffic to the ALB and EC2 instances.
Application Load Balancer (ALB)	Receives HTTP traffic from users and distributes it to healthy EC2 instances.
Target Group	Maintains the backend EC2 targets and performs HTTP health checks.
Auto Scaling Group (ASG)	Maintains the desired EC2 capacity and automatically replaces terminated instances.
Amazon CloudWatch	Provides CPU utilization metrics used by the Target Tracking scaling policy.
Availability Zones	Provide multi-AZ deployment for improved availability and resilience.

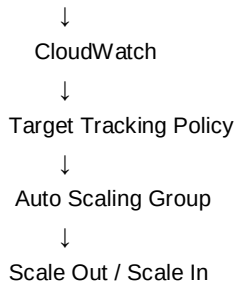
Service Relationship

The services work together as a chain:



CloudWatch monitors the EC2 workload and provides the metric used by the Auto Scaling policy:

EC2 CPU Utilization



The combination of these services allows the application to remain available even when an individual EC2 instance fails.

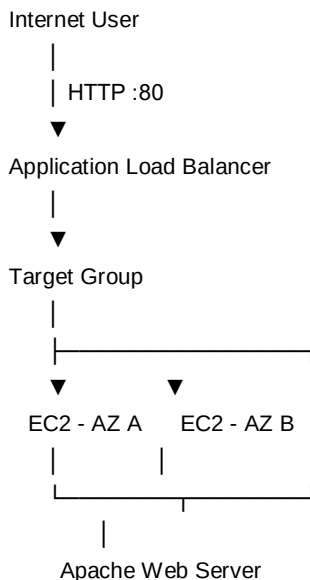
4. Architecture Design

The architecture was designed around four main principles:

- **High Availability** — EC2 instances are distributed across multiple Availability Zones.
- **Load Balancing** — incoming traffic is handled by an Application Load Balancer instead of directly accessing EC2 instances.
- **Scalability** — the Auto Scaling Group can increase or decrease the number of EC2 instances based on CPU utilization.
- **Self-Healing** — if an EC2 instance is terminated or becomes unavailable, the Auto Scaling Group launches a replacement.

4.1 Traffic Flow

The application follows this request path:



Users never need to connect directly to an individual EC2 instance. The ALB acts as the single entry point for the application.

4.2 Security Flow

Two Security Groups were created:

SG-ALB

Inbound:

HTTP :80 → 0.0.0.0/0

Outbound:

All traffic

This allows users from the internet to reach the ALB.

SG-EC2

Inbound:

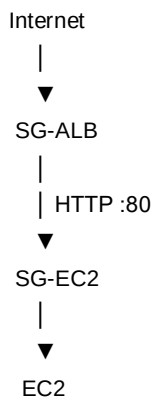
HTTP :80 → SG-ALB

Outbound:

All traffic

Instead of allowing HTTP traffic from the entire internet, the EC2 Security Group references **SG-ALB** as the source.

Therefore:

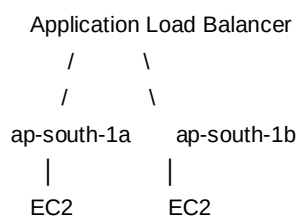


This prevents users from directly accessing the application instances through port 80.

4.3 Multi-AZ Design

The ALB and EC2 instances were configured across two Availability Zones:

AWS Region: ap-south-1



This provides redundancy at the Availability Zone level. If an individual instance becomes unavailable, the remaining healthy instance can continue serving traffic while the Auto Scaling Group launches a replacement.

4.4 Scaling and Self-Healing Flow

The Auto Scaling Group was configured with:

Minimum capacity: 1

Desired capacity: 2

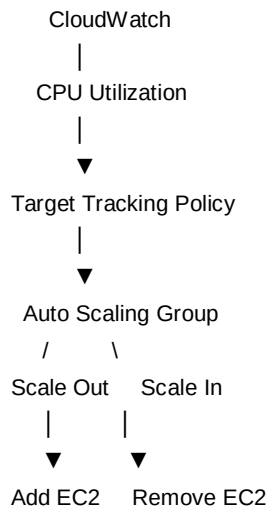
Maximum capacity: 4

A Target Tracking policy was later configured using:

Metric: Average CPU Utilization

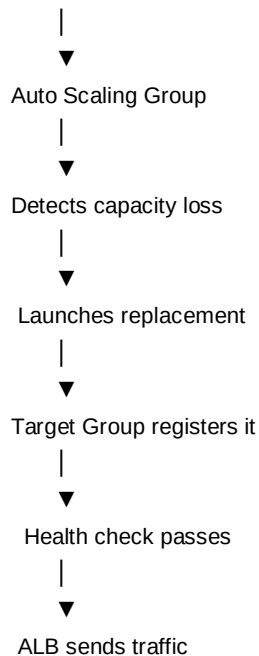
Target: 50%

The overall behavior is:



For failure recovery:

EC2 instance fails/terminates



This architecture was later verified experimentally by terminating an active EC2 instance and observing the Auto Scaling Group launch a replacement that became healthy in the Target Group.

5. Security Groups Configuration

Security Groups were created before launching the application infrastructure to control traffic between the internet, the Application Load Balancer, and the EC2 instances.

5.1 SG-ALB

The Security Group attached to the Application Load Balancer allows HTTP traffic from the internet.

Inbound Rules

HTTP

Port: 80

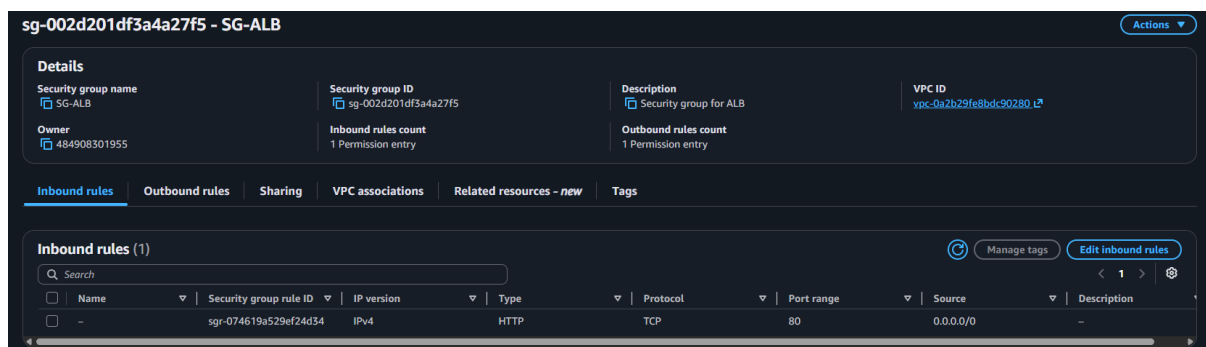
Source: 0.0.0.0/0

Outbound Rules

All traffic

This allows users to access the application through the ALB using HTTP.

Screenshot — SG-ALB configuration



5.2 SG-EC2

The EC2 Security Group was intentionally configured differently.

Inbound Rules

HTTP

Port: 80

Source: SG-ALB

Outbound Rules

All traffic

The source of the HTTP rule is **SG-ALB itself**, rather than 0.0.0.0/0.

This means an EC2 instance accepts HTTP traffic only when it originates from a resource associated with SG-ALB.

Internet

|

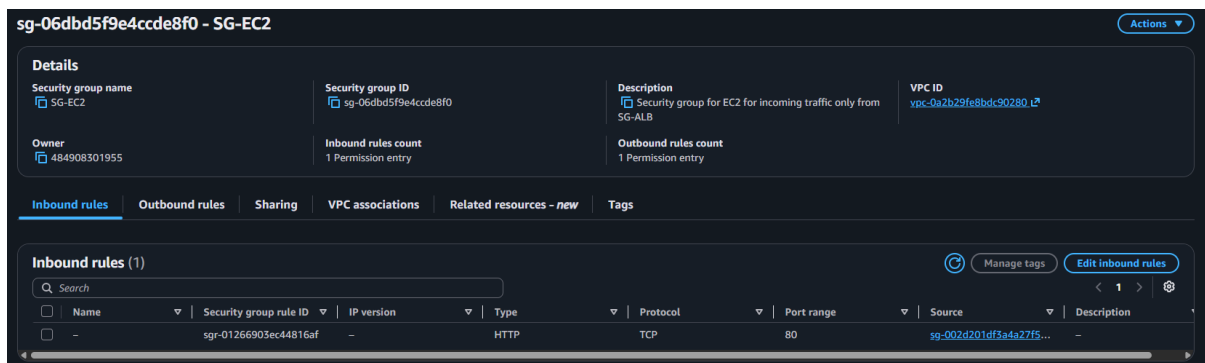
```

| HTTP :80
▼
SG-ALB
|
| HTTP :80
▼
SG-EC2
|
▼
EC2

```

This prevents direct HTTP access to the EC2 instances from the public internet and ensures that application traffic enters through the ALB.

Screenshot — SG-EC2 configuration



5.3 Temporary SSH Access for Testing

During the testing phase, a temporary SSH rule was added to SG-EC2 so that the EC2 instance could be accessed for verification and controlled testing.

SSH

Port: 22

Source: <Public IP>/32

The source was restricted to the specific public IP rather than allowing SSH from the entire internet.

After completing the testing, the project resources were cleaned up.

Note: SSH was used only as a temporary testing mechanism and was not part of the intended application traffic path.

6. Launch Template Configuration

The Launch Template was created to provide a **consistent and reusable configuration** for every EC2 instance launched by the Auto Scaling Group.

Instead of manually configuring each EC2 instance, the ASG uses this template whenever it needs to launch an instance.

6.1 Launch Template Configuration

The Launch Template was configured with:

Launch Template Name: web-server-lt

AMI: Amazon Linux 2023

Architecture: 64-bit x86

Instance Type: t3.micro

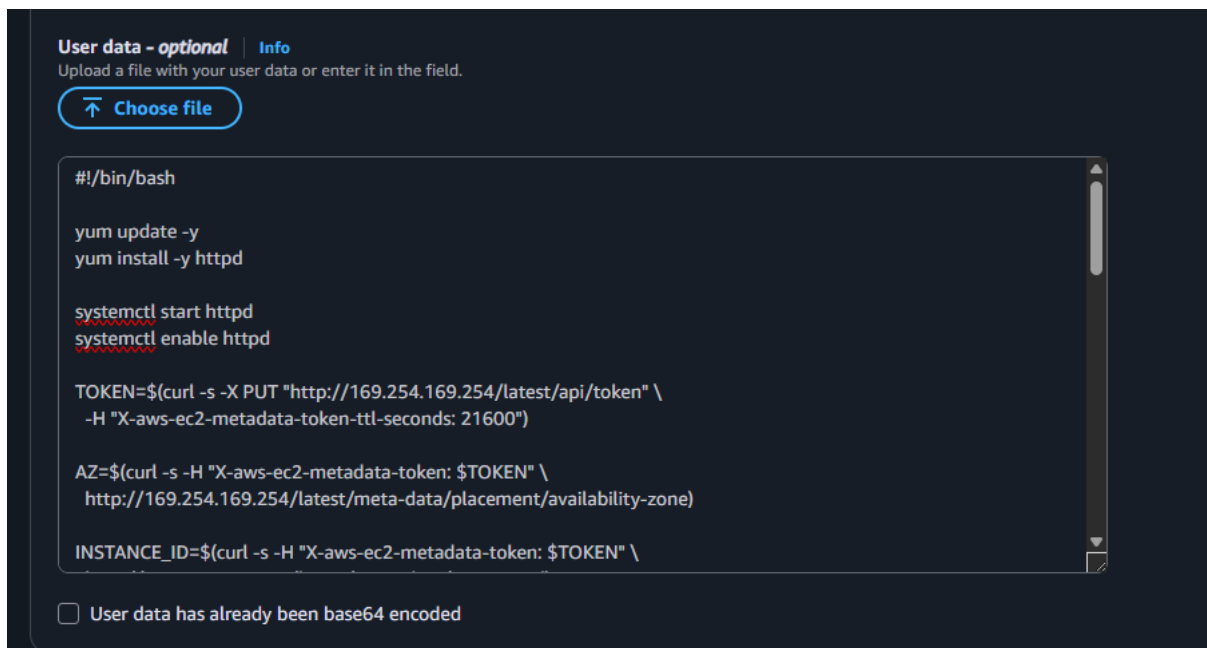
Key Pair: lab-ec2

Security Group: SG-EC2

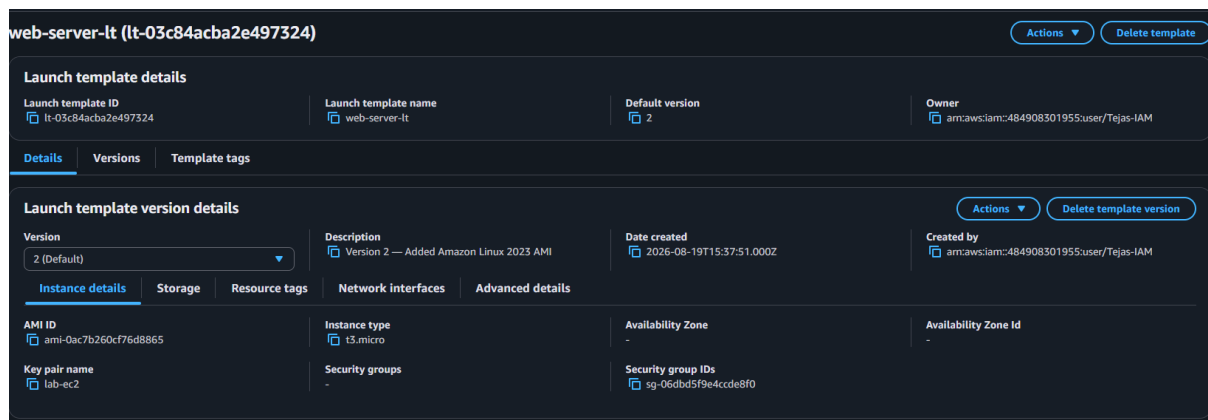
The Launch Template was initially created as Version 1. During verification, the AMI configuration was corrected and **Version 2** was created with the Amazon Linux 2023 AMI.

Version 2 was then made the **default version** used by the Auto Scaling Group.

Screenshot — Launch Template User Data



Screenshot — Final Launch Template configuration



6.2 User Data Automation

User Data was used to automatically configure the EC2 instance when it launches.

The script performs the following tasks:

1. Updates the system packages.
2. Installs Apache (httpd).
3. Starts the Apache service.
4. Enables Apache to start automatically.
5. Retrieves instance metadata using the EC2 Instance Metadata Service.
6. Retrieves:
 - o Instance ID
 - o Availability Zone
 - o AWS Region
7. Generates a custom HTML webpage containing the instance information.

The generated webpage was intentionally made specific to the project:

AWS Highly Available Web Application

Built by Tejas Shinkar

ELB + Auto Scaling + EC2 + CloudWatch

Instance ID: ...

Availability Zone: ...

Region: ...

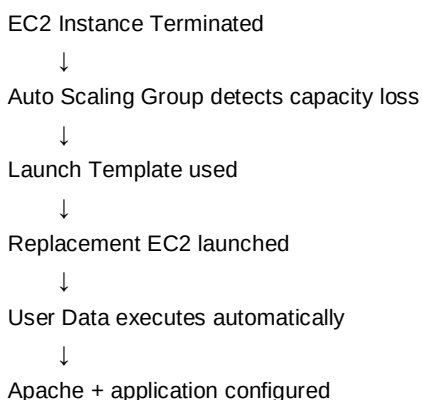
This allowed the ALB's traffic distribution to be visually verified later. When requests reached different EC2 instances, the **Instance ID and Availability Zone displayed on the webpage changed**.

6.3 Why Launch Templates Were Used

The Launch Template ensures that every EC2 instance launched by the ASG receives the same basic configuration.

This was particularly important during the self-healing test.

When an instance was terminated:



This allowed the replacement instance to become a functional backend without manually configuring it.

7. Target Group Configuration

A **Target Group** acts as the backend pool for the Application Load Balancer. It contains the EC2 instances that can receive traffic and performs health checks to determine whether those instances are healthy.

7.1 Target Group Configuration

The Target Group was configured as:

Target Type: Instances
 Target Group Name: web-server-tg
 Protocol: HTTP
 Port: 80
 VPC: Default VPC
 Health Check: HTTP
 Health Check Path: /
 Health Check Port: Traffic Port
 Interval: 30 seconds
 Timeout: 5 seconds
 Healthy Threshold: 2
 Unhealthy Threshold: 2
 Success Code: 200

Screenshot — Target Group configuration

The screenshot displays the AWS Management Console interface for configuring a Target Group. It is divided into two main sections: 'Step 1: Target group details' and 'Step 2: Register targets'.

Step 1: Target group details (with an 'Edit' button):

- Target group details:**
 - Name: web-server-tg
 - VPC: vpc-0a2b29fe8bdc90280
 - Target type: Instance
 - IP address type: IPv4
 - Protocol : Port: HTTP: 80
 - Protocol version: HTTP1
- Health check details:**
 - Health check protocol: HTTP
 - Health check path: /
 - Health check port: traffic-port
 - Interval: 30 seconds
 - Timeout: 5 seconds
 - Healthy threshold: 2
 - Unhealthy threshold: 2
 - Success codes: 200

Step 2: Register targets (with an 'Edit' button):

- Targets (0):** A table with columns for Instance ID, Name, Port, and Zone. It currently shows 'No targets added'.
- Server-side tasks and status:** A section with a right-pointing arrow and the text: 'After completing and submitting the above steps, all server-side tasks and their statuses become available for monitoring.'

At creation time, the Target Group showed:

Targets: 0

This was intentional.

The EC2 instances were **not manually registered**. Instead, the Auto Scaling Group was configured to automatically register newly launched EC2 instances with web-server-tg.

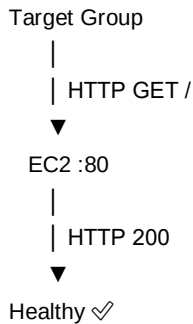
7.2 Health Checks

The Target Group continuously checks the health of registered EC2 instances by requesting:

HTTP://<EC2-IP>/

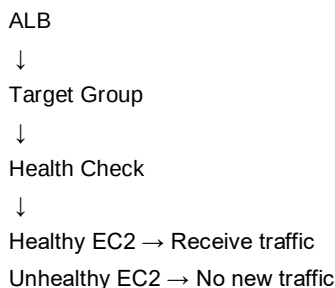
A successful HTTP response with status code 200 indicates that the application is responding correctly.

The flow is:



If an instance fails the configured health checks, the ALB stops routing new traffic to that instance.

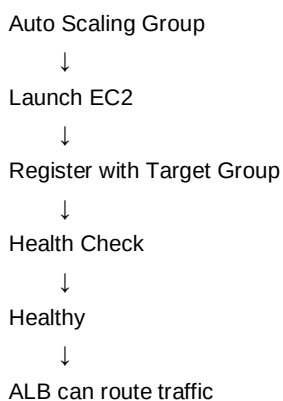
The Target Group therefore provides an important layer between the ALB and EC2 instances:



7.3 Automatic Target Registration

Once the Auto Scaling Group launched the EC2 instances, they were automatically registered with the Target Group.

The final flow was:



This becomes particularly important during self-healing. When the ASG launched a replacement instance after an instance was terminated, the replacement was automatically registered with the Target Group and eventually reached a **Healthy** state.

8. Application Load Balancer Configuration

The Application Load Balancer was configured as the **public entry point** for the web application.

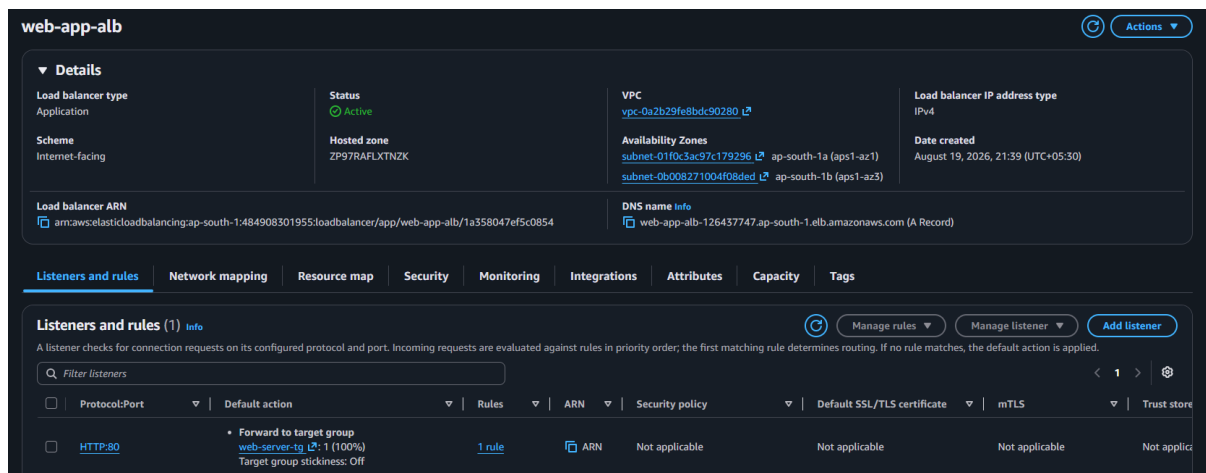
8.1 ALB Configuration

The following configuration was used:

Name: web-app-alb
 Type: Application Load Balancer
 Scheme: Internet-facing
 IP Address Type: IPv4
 VPC: Default VPC
 Availability Zones:
 ap-south-1a
 ap-south-1b
 Security Group: SG-ALB

The ALB was deployed across **two Availability Zones** to provide availability at the load-balancer layer.

Screenshot — Application Load Balancer configuration



8.2 Listener Configuration

The ALB was configured with an HTTP listener:

Protocol: HTTP
 Port: 80

Default Action:
 Forward traffic → web-server-tg

Therefore, when a user accesses the ALB DNS name:

http://<ALB-DNS-name>

the request follows:

```

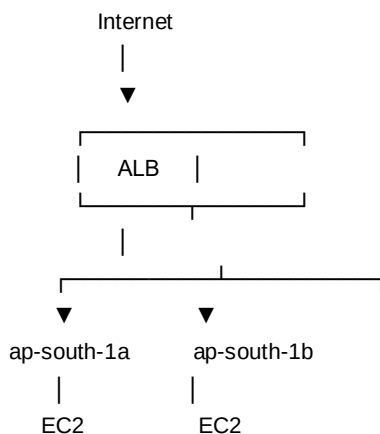
User
↓
ALB :80
↓
web-server-tg
↓
Healthy EC2 :80

```

The ALB does not directly select an arbitrary EC2 instance. It forwards traffic to the **healthy targets registered in the Target Group**.

8.3 ALB and Multi-AZ Architecture

The ALB was configured using subnets in two different Availability Zones:



This allows the ALB to route requests to healthy backend instances across Availability Zones.

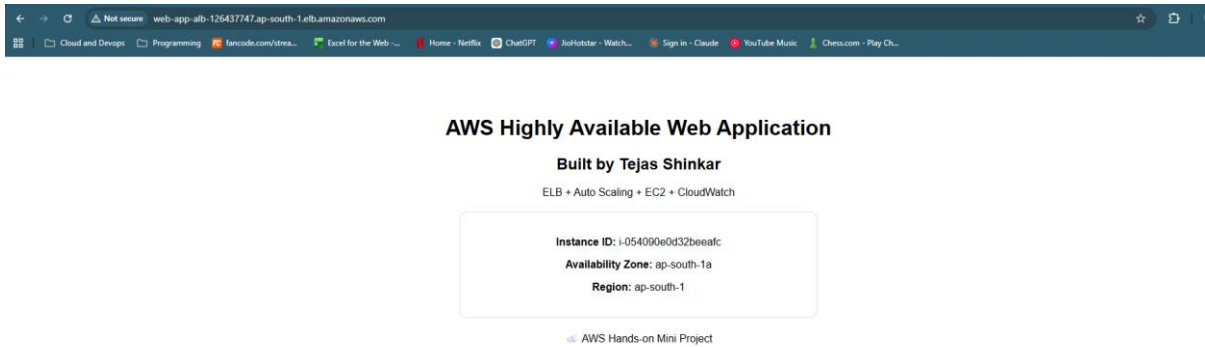
8.4 ALB Verification

After the ALB became active and the EC2 instances were healthy, the ALB DNS name was accessed through a web browser.

The custom application page displayed the instance-specific information generated by the Launch Template's User Data script.

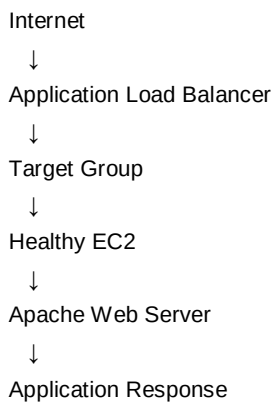
By refreshing the ALB endpoint multiple times while both instances were healthy, requests were observed reaching different EC2 instances.

The changing **Instance ID and Availability Zone** provided visible evidence that the ALB was distributing requests across the backend instances.



This

verified the complete request path:



9. Auto Scaling Group Configuration

The Auto Scaling Group was responsible for managing the EC2 instance fleet and maintaining the required application capacity.

9.1 ASG Configuration

The Auto Scaling Group was created with:

Name: web-app-asg
 Launch Template: web-server-1t
 Launch Template Version: Latest / Version 2

Desired Capacity: 2
 Minimum Capacity: 1
 Maximum Capacity: 4

The ASG used the same VPC and the two subnets selected for the application infrastructure:

ap-south-1a
 ap-south-1b

This allowed the EC2 instances to be distributed across multiple Availability Zones.

Screenshot — Auto Scaling Group configuration

Step 1: Choose launch template

Edit

Group details

Auto Scaling group name
web-app-asg

Launch template

Launch template	Version	Description
web-server-1t ↗ lt-03c84acba2e497324	Latest	Version 2 — Added Amazon Linux 2023 AMI

Step 2: Choose instance launch options

Edit

Network

VPC
vpc-0a2b29fe8bdc90280 [↗](#)

Availability Zones and subnets

Availability Zone	Subnet	Subnet CIDR range
aps1-az1 (ap-south-1a)	subnet-01f0c3ac97c179296 ↗	172.31.32.0/20
aps1-az3 (ap-south-1b)	subnet-0b008271004f08ded ↗	172.31.0.0/20

Availability Zone distribution
Balanced best effort

Capacity Reservation preference

Preference	Capacity Reservation IDs	Resource Groups
Default	-	-

Instance type requirements

This Auto Scaling group will adhere to the launch template.

Step 3: Integrate with other services

Edit

Load balancing

Load balancer 1

Name	Type	Target group
web-app-alb ↗	Application/HTTP	web-server-tg ↗

VPC Lattice integration options

VPC Lattice target groups
-

Application Recovery Controller (ARC) zonal shift

ARC zonal shift
Disabled

Health checks

Health check type	Health check grace period
EC2, ELB	300 seconds

Step 4: Configure group size and scaling policies

Edit

Group size

Desired capacity	Desired capacity type
1	Units (number of instances)

Scaling

Minimum desired capacity	Maximum desired capacity
1	1

Target tracking policy
-

Instance maintenance policy

Replacement behavior	Min healthy percentage	Max healthy percentage
No policy	-	-

Additional settings

Instance scale-in protection	Monitoring	Default instance warmup
Disabled	Disabled	Disabled

Placement group	Auto Scaling group deletion protection
-	None (default)

9.2 Load Balancer Integration

The ASG was attached to the existing Target Group:

web-server-tg

This means EC2 instances launched by the ASG were automatically registered with the Target Group.

The ASG was configured to use **ELB health checks** in addition to EC2 health checks.

EC2 Health Check

+

ELB Health Check

↓

Auto Scaling Group

The ELB health check is important because an EC2 instance can be running at the infrastructure level while the application itself may not be responding correctly.

The ASG therefore works with the load balancer's health information to help maintain healthy application capacity.

9.3 Health Check Grace Period

A **300-second health check grace period** was configured.

This gives a newly launched EC2 instance time to:

- Boot the operating system.
- Execute the User Data script.
- Install Apache.
- Start the web server.
- Become ready to respond to the Target Group health checks.

This is particularly important because the application configuration is performed automatically through User Data during instance startup.

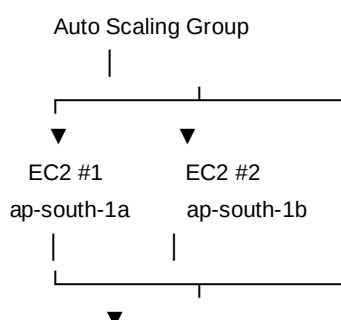
9.4 Initial Deployment

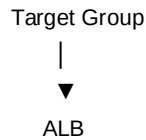
When the ASG was created with:

Desired Capacity = 2

it automatically launched two EC2 instances using the Launch Template.

The resulting architecture was:





Both instances were successfully registered with the Target Group and reached a **Healthy** state.

Screenshot — ASG-launched EC2 instances and Target Group health

web-app-asg Capacity overview

arn:aws:autoscaling:ap-south-1:484908301955:autoScalingGroup:786831e8-58be-4e59-8518-3dfb9a41f22d:autoScalingGroupName/web-app-asg

Desired capacity: 2 | Scaling limits: 1 - 4 | Desired capacity type: Units (number of instances) | Status: At desired capacity

Date created: Wed Aug 19 2026 21:53:27 GMT+0530 (India Standard Time)

Details | Integrations | Automatic scaling | **Instance management** | Instance refresh | Activity | Monitoring | Tags

Instances (2)

Instance ID	Lifecycle	Instance type	Weighted capacity	Launch templat...	Availability Zone	Health status	Protected from
i-054090e0d32beaafc	InService	t3.micro	-	web-server-lt Versio	aps1-az1 (ap-south-...	Healthy	
i-0dcb27554720d2f95	InService	t3.micro	-	web-server-lt Versio	aps1-az3 (ap-south-...	Healthy	

web-server-tg

Details

arn:aws:elasticloadbalancing:ap-south-1:484908301955:targetgroup/web-server-tg/09345073e75fc73e

Target type: Instance | Protocol: Port: HTTP: 80 | Protocol version: HTTP1 | VPC: vpc-0a2b29fe8bdc90280

IP address type: IPv4 | Load balancer: web-app-alb

2 Total targets | 2 Healthy | 0 Unhealthy | 0 Anomalous

0 Unused | 0 Initial | 0 Draining

► Distribution of targets by Availability Zone (AZ)

Targets | **Monitoring** | Health checks | Attributes | Tags

Registered targets (2)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anomaly d
i-054090e0d32beaafc	-	80	ap-south-1a (a...	Healthy	-	No override	No override is cure...	August 19...	Normal
i-0dcb27554720d2f95	-	80	ap-south-1b (a...	Healthy	-	No override	No override is cure...	August 19...	Normal

This verified that the Launch Template, Auto Scaling Group, Target Group, and health-check configuration were working together correctly.

10. Target Tracking Auto Scaling Policy

After verifying that the initial EC2 deployment and ALB routing were working correctly, a **Target Tracking Scaling Policy** was configured for the Auto Scaling Group.

10.1 Scaling Policy Configuration

The policy was configured with:

Policy Type: Target Tracking Scaling
 Policy Name: cpu-target-tracking
 Metric: Average CPU Utilization
 Target Value: 50%
 Instance Warmup: 300 seconds
 Scale In: Enabled
 Scale Out: Enabled

Screenshot — Target Tracking Scaling Policy

Create dynamic scaling policy

Policy type
 Target tracking scaling

Scaling policy name
 Target Tracking Policy

Metric type Info
 Monitored metric that determines if resource utilization is too low or high. If using EC2 metrics, consider enabling detailed monitoring for better scaling performance.
 Average CPU utilization

Target value
 50

Instance warmup Info
 300 seconds

☐ Disable scale in to create only a scale-out policy

[Cancel](#) [Create](#)

web-app-asg

web-app-asg Capacity overview [Edit](#)

arn:aws:autoscaling:ap-south-1:484908301955:autoScalingGroup:786831e8-58be-4e59-8518-3dfb9a41f22d:autoScalingGroupName/web-app-asg

Desired capacity 1	Scaling limits 1 - 4	Desired capacity type Units (number of instances)	Status At desired capacity
------------------------------	--------------------------------	---	--------------------------------------

Date created
Wed Aug 19 2026 21:53:27 GMT+0530 (India Standard Time)

Details | **Integrations** | **Automatic scaling** | **Instance management** | **Instance refresh** | **Activity** | **Monitoring** | **Tags**

Scaling policies resize your Auto Scaling group to meet changes in demand. With reactive dynamic scaling policies, you can track specific CloudWatch metrics and take action when the CloudWatch alarm threshold is met. Use predictive scaling policies along with dynamic scaling policies in the following situations: when your application demand changes quickly, but with a recurring pattern, or when your EC2 instances require more time to initialize.

Dynamic scaling policies (1) Info [Actions](#) [Create dynamic scaling policy](#)

Target Tracking Policy ☐

Policy type
Target tracking scaling

Enabled or disabled
Enabled

Execute policy when
As required to maintain Average CPU utilization at 50

Take the action
Add or remove capacity units as required

Instances need
300 seconds to warm up before including in metric

Scale in
Enabled

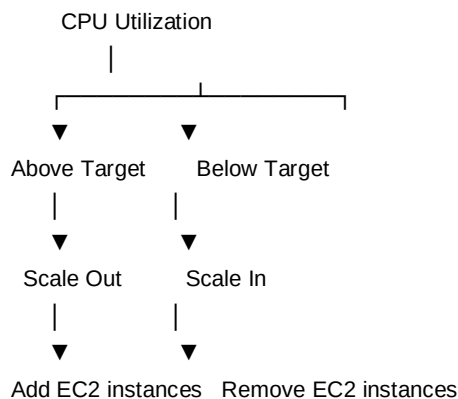
10.2 How Target Tracking Works

The purpose of Target Tracking is to maintain the average CPU utilization of the Auto Scaling Group around the configured target.

In this project, the target was:

Average CPU Utilization = 50%

The general behavior is:



The Auto Scaling Group uses CloudWatch metrics to determine when the application capacity needs to change.

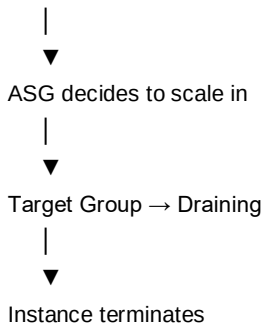
10.3 Scale-In Behavior Observed

After enabling the Target Tracking policy, the workload on the instances was low.

The ASG therefore reduced the desired capacity from the initial deployment level.

The instance being removed entered the **Draining** state in the Target Group before termination.

Healthy EC2



This demonstrated that scale-in does not simply remove an instance from active traffic immediately. The load-balancing layer handles the target transition before the instance is terminated.

Screenshot — Instance draining / scale-in

Instance ID	Name	Port	Zone	Health status	Health status details	Administrative o...	Override details	Launch...	Anomaly d
i-054090e0d32beaefc	-	80	ap-south-1a (a...	Draining	Target deregistration I...	No override	No override is curre...	August 19...	Normal
i-0dc27554720d2f95	-	80	ap-south-1b (a...	Healthy	-	No override	No override is curre...	August 19...	Normal

10.4 Resulting Capacity

After the scale-in operation completed, the environment reached:

Desired Capacity: 1

Healthy Targets: 1

This confirmed that the Auto Scaling Group was able to automatically reduce the number of running instances when additional capacity was not required.

The scaling policy therefore demonstrated both sides of the desired behavior:

- **Scale out** when workload increases.
 - **Scale in** when workload decreases.
-

11. Application Verification and Testing

After deploying the infrastructure, the application was tested at multiple levels to verify that the EC2 configuration, load balancer, and backend routing were functioning correctly.

11.1 EC2 Web Server Verification

SSH access was temporarily enabled from the test environment's public IP for instance-level verification.

Inside the EC2 instance, the Apache web server was checked using:

```
sudo systemctl status httpd
```

The service was confirmed to be:

```
active (running)
```

This verified that the Apache web server installed through the Launch Template's User Data script was running correctly.

The application was then tested locally using:

```
curl localhost
```

The command returned the custom HTML application, confirming that the EC2 instance was successfully serving the webpage.

```
[ec2-user@ip-172-31-15-62 ~]$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Tejas Shinkar| AWS ELB & Auto Scaling Project</title>
<style>
  body {
    font-family: Arial;
    text-align: center;
    padding: 60px;
  }
  .info {
    margin: 20px auto;
    max-width: 500px;
    padding: 20px;
    border: 1px solid #ddd;
    border-radius: 10px;
  }
</style>
</head>
<body>
<h1>AWS Highly Available Web Application</h1>
<h2>Built by Tejas Shinkar</h2>

<p>ELB + Auto Scaling + EC2 + CloudWatch</p>

<div class="info">
  <p><b>Instance ID:</b> i-0dcb27554720d2f95</p>
  <p><b>Availability Zone:</b> ap-south-1b</p>
  <p><b>Region:</b> ap-south-1</p>
</div>

<p>👉 AWS Hands-on Mini Project</p>
</body>
</html>
[ec2-user@ip-172-31-15-62 ~]$ Connection to ec2-13-201-130-185.ap-south-1.compute.amazonaws.com closed by remote host.
Connection to ec2-13-201-130-185.ap-south-1.compute.amazonaws.com closed.
```

Screenshot — Local application verification

11.2 ALB Application Verification

The application was then accessed through the **Application Load Balancer DNS name** rather than directly through the EC2 instance.

The webpage displayed:

AWS Highly Available Web Application

Built by Tejas

ELB + Auto Scaling + EC2 + CloudWatch

Instance ID: ...

Availability Zone: ...

Region: ...

This verified the complete application path:

Browser



ALB



Target Group



EC2



Apache



Application

11.3 Load Balancing Verification

When both backend instances were healthy, the ALB was accessed repeatedly through its DNS endpoint.

Refreshing the application resulted in different **Instance IDs and Availability Zones** being displayed.

For example:

Request 1 → EC2 in ap-south-1a

Request 2 → EC2 in ap-south-1b

Request 3 → EC2 in ap-south-1a

This provided a simple visual demonstration that the ALB was distributing requests between healthy backend instances.

The Instance ID displayed on the webpage was generated dynamically from EC2 instance metadata by the User Data script, making the change directly observable.

11.4 Target Group Health Verification

The Target Group was monitored during deployment and after instance replacement.

A healthy target appeared as:

```
Target
↓
HTTP :80
↓
GET /
↓
HTTP 200
↓
Healthy
```

This confirmed that the ALB was not simply checking whether the EC2 machine was running; it was checking whether the web application was responding successfully.

11.5 Testing Summary

The following components were successfully verified:

Test	Result
Apache service running	✓
Local <code>curl localhost</code>	✓
ALB DNS access	✓
Target Group health checks	✓
ALB → EC2 routing	✓

Test	Result
Instance ID/AZ visibility	✓
Multi-AZ backend deployment	✓
Auto Scaling integration	✓

The testing confirmed that the application infrastructure was functioning as an integrated system rather than as a collection of independently configured AWS services.

12. Self-Healing and Failure Simulation

One of the most important parts of the project was testing what happens when an EC2 instance unexpectedly disappears.

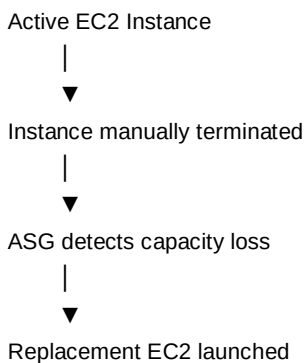
Instead of only verifying that the infrastructure works under normal conditions, an active EC2 instance was deliberately **terminated** to simulate an instance failure.

12.1 Failure Simulation

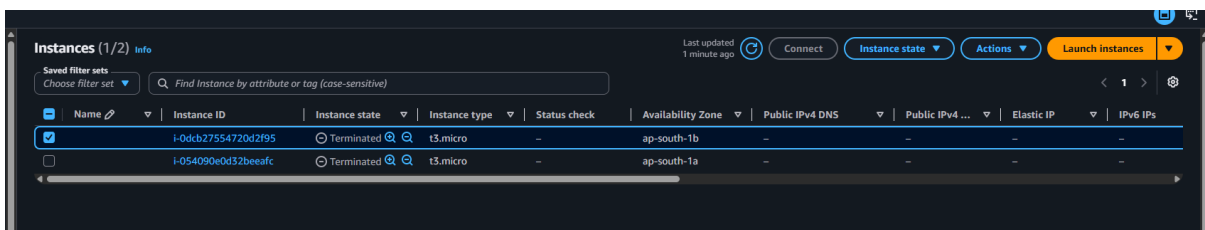
An EC2 instance managed by the Auto Scaling Group was manually terminated.

Before the failure, the ASG was responsible for maintaining the required application capacity.

The sequence was:



Screenshot — Instance termination / failure simulation



12.2 Automatic Replacement

After the instance was terminated, the Auto Scaling Group detected that the number of running instances had fallen below its required capacity.

The ASG automatically launched a replacement EC2 instance using the configured Launch Template.

The replacement instance therefore received the same:

- Amazon Linux 2023 AMI
- Instance type
- Security Group
- Key pair configuration
- User Data automation
- Apache web server configuration

The replacement instance successfully completed its EC2 status checks.

12.3 Target Group Registration

After the replacement instance became operational, the Auto Scaling Group automatically registered it with the existing Target Group.

The Target Group then performed its configured HTTP health check:

Replacement EC2



HTTP GET /

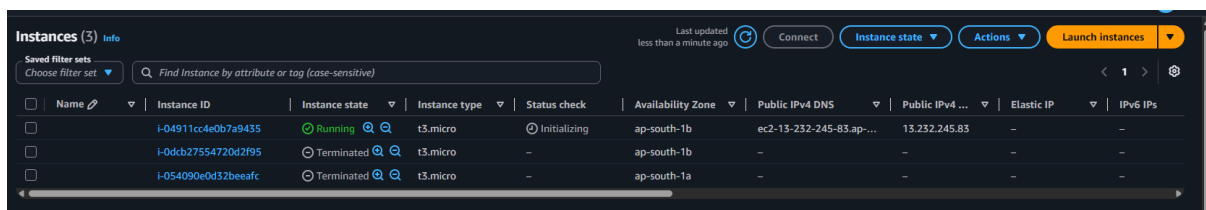


HTTP 200



Healthy

The replacement instance eventually appeared as **Healthy** in web-server-tg.



Screenshot — Replacement EC2 instance launched

The project documentation records this sequence: the original server was terminated, a new instance was automatically added, and the new instance became healthy in the Target Group.

12.4 Self-Healing Result

The complete recovery sequence was successfully demonstrated:

EC2 Instance



Failure / Termination

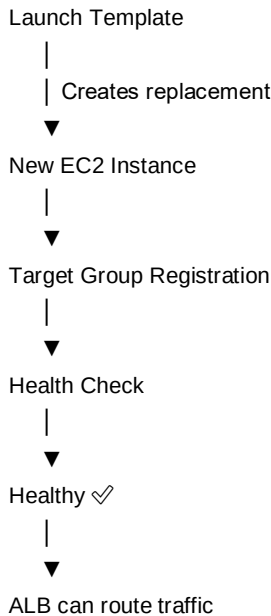


Auto Scaling Group



Detects capacity loss





This demonstrated the **self-healing capability** of the architecture.

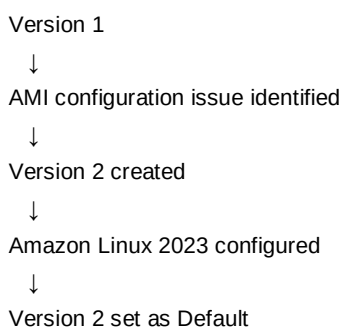
13. Troubleshooting & Issues Encountered

During implementation, a few configuration issues were encountered and resolved. Documenting these is useful because they reflect realistic AWS troubleshooting rather than a perfectly linear deployment.

13.1 Launch Template AMI Configuration

The initial Launch Template version did not contain the expected AMI configuration.

A new Launch Template version was created with the **Amazon Linux 2023 AMI**, and Version 2 was made the default version.



This ensured that instances launched by the Auto Scaling Group used the correct AMI.

13.2 SSH Connectivity

The initial EC2 Security Group did not allow SSH because the intended application architecture allowed HTTP traffic only from the ALB Security Group.

When instance-level testing was required, SSH access was temporarily added:

SSH :22
Source: <Public IP>/32

The source was restricted to the testing machine's public IP rather than exposing SSH to the entire internet.

This allowed:

SSH → EC2

for troubleshooting and application verification while keeping the rule narrowly scoped.

13.3 Auto Scaling Capacity Change

The initial ASG deployment started with:

Desired: 2

Minimum: 1

Maximum: 4

After the Target Tracking policy was enabled, the low workload caused the ASG to scale in.

The environment eventually reached:

Desired: 1

Healthy Targets: 1

This explained why subsequent ALB refreshes showed the same Instance ID—the ALB had only one healthy backend available at that point.

13.4 EC2 Instance Connect / SSH Timeout

An initial SSH connection attempt failed because the EC2 Security Group did not allow inbound TCP port 22.

The issue was traced to the security configuration rather than the application itself.

A temporary SSH rule restricted to the public IP was added, after which SSH connectivity succeeded.

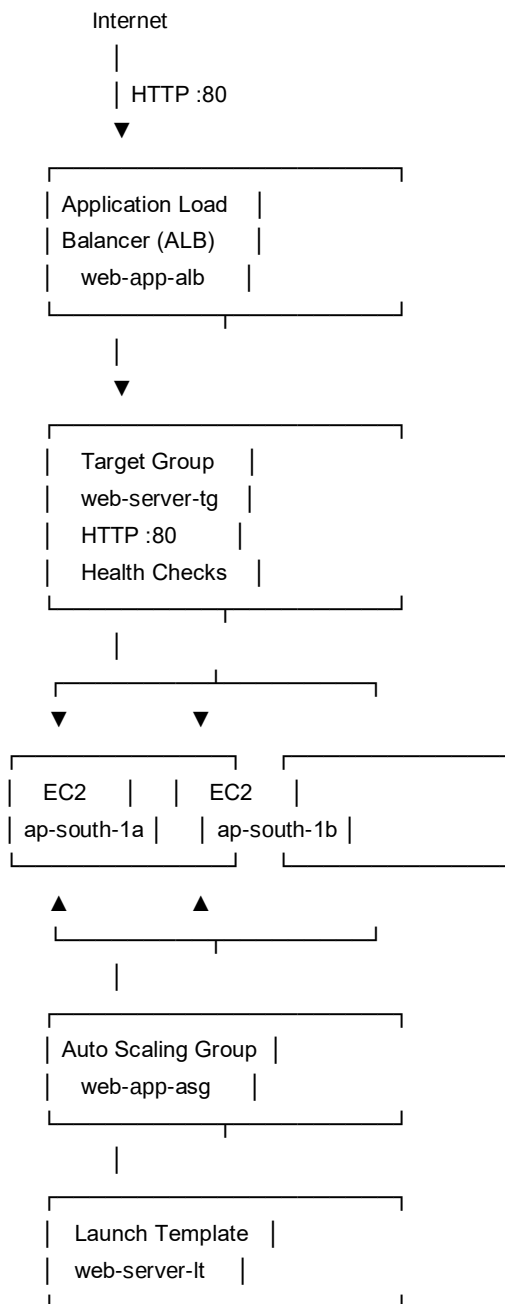
The rule was intended only for testing and was removed during cleanup.

These troubleshooting steps were important because they demonstrated that AWS infrastructure deployment involves validating the interaction between **Security Groups, Launch Templates, ASG configuration, Target Groups, and application-level services**, rather than simply checking whether individual resources were created successfully.

14. Final Architecture and End-to-End Flow

After completing the configuration and testing, the final architecture consisted of an Internet-facing Application Load Balancer, a Target Group, EC2 instances managed by an Auto Scaling Group, and a reusable Launch Template.

14.1 Final Architecture



14.2 End-to-End Request Flow

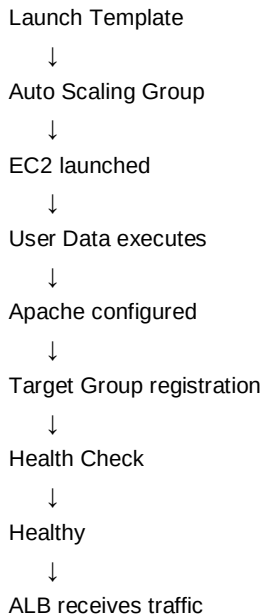
When a user accesses the application:

1. User sends HTTP request
- ↓
2. Request reaches Internet-facing ALB
- ↓
3. ALB forwards request to web-server-tg
- ↓
4. Target Group checks available targets
- ↓
5. Request is sent to a Healthy EC2 instance
- ↓

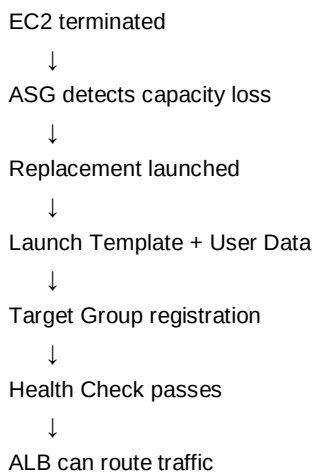
6. Apache serves the application
- ↓
7. Response travels back through the ALB
- ↓
8. User receives the webpage

14.3 Instance Lifecycle

The Auto Scaling Group controls the lifecycle of the backend EC2 instances:

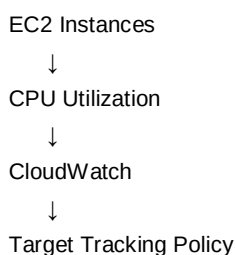


If an instance is removed:



14.4 Scaling Flow

The scaling policy adds another control loop:





Auto Scaling Group



Scale Out / Scale In

This creates an architecture that combines:

Load Balancing + Multi-AZ Availability + Health Checks + Auto Scaling + Self-Healing

Rather than relying on a single EC2 instance.

15. Key Learnings

This project helped connect several AWS concepts that are often learned separately and demonstrated how they work together in a production-style architecture.

15.1 Load Balancing

An Application Load Balancer provides a single entry point for users while distributing requests across healthy backend instances.

Instead of users accessing individual EC2 instances directly:

User → EC2

the architecture uses:

User → ALB → Target Group → EC2

This provides a cleaner and more resilient application entry point.

15.2 Target Groups

A Target Group is more than simply a list of EC2 instances.

It provides:

- Backend target registration
- Health checks
- Health status
- Integration between the ALB and backend instances

An instance that is unhealthy should not continue receiving application traffic.

15.3 Auto Scaling

Auto Scaling allows infrastructure capacity to change according to application demand.

The important distinction learned during the project was:

Scaling



Change the number of instances

Self-Healing



Replace failed instances

The same Auto Scaling Group can provide both capabilities.

15.4 Launch Templates and Automation

The Launch Template acts as a repeatable blueprint for EC2 instances.

Using User Data made the instance configuration automatic.

Instead of manually:

Launch EC2



Install Apache



Start Apache



Create webpage

the Launch Template automated the configuration whenever a new instance was launched.

This became particularly valuable during the self-healing test because the replacement instance could configure itself automatically.

15.5 Health Checks

The project demonstrated the difference between an instance being **running** and an application being **healthy**.

An EC2 instance may be operational at the infrastructure level while its web application is unavailable.

The Target Group's HTTP health check provided an application-level check:

EC2 running

≠

Application healthy

The application needed to successfully respond to the configured HTTP health check.

15.6 Multi-AZ Architecture

Deploying resources across multiple Availability Zones reduces dependence on a single AZ.

In this project:

ap-south-1a



EC2

ap-south-1b



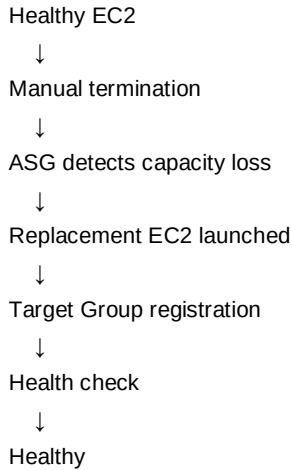
EC2

The ALB could route traffic to healthy instances across these Availability Zones.

15.7 Self-Healing

The most important practical learning was observing self-healing rather than just reading about it.

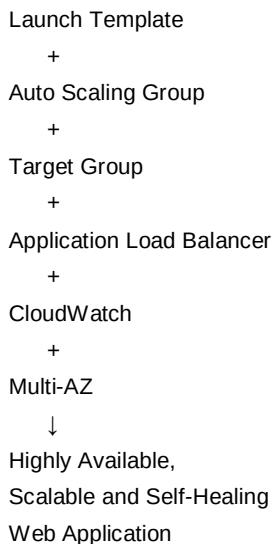
The experiment was:



The replacement instance became healthy without manually recreating the server configuration.

15.8 Production Architecture Is About Service Interaction

The biggest takeaway from the project was that individual AWS services become significantly more powerful when combined.



The project therefore provided practical experience not only with individual AWS services, but with **designing and validating how multiple services interact to provide a resilient application infrastructure**.

16. Cleanup and Final State

After completing all testing, the AWS resources created specifically for this project were removed to avoid unnecessary ongoing costs.

Cleanup was performed in dependency order.

16.1 Auto Scaling Group

The Auto Scaling Group `web-app-asg` was deleted first.

Deleting the ASG also terminated the EC2 instances that it was managing.

Auto Scaling Group



EC2 instances terminated

This prevented the ASG from launching new replacement instances while the remaining infrastructure was being removed.

16.2 Application Load Balancer

After confirming that the ASG and its EC2 instances were gone, the Application Load Balancer was deleted:

`web-app-alb`

16.3 Target Group

The Target Group was then deleted:

`web-server-tg`

16.4 Launch Template

The Launch Template created specifically for the project was removed:

`web-server-lt`

The Launch Template itself does not create running instances; it is simply a reusable configuration. It was deleted after the ASG and instances were removed.

16.5 Security Groups

The project-specific Security Groups were deleted after confirming that no remaining resources were using them:

`SG-ALB`

`SG-EC2`

The temporary SSH rule used during testing was therefore no longer present after the project cleanup.

16.6 Final Resource State

The project-specific infrastructure was completely cleaned up:

Auto Scaling Group → Deleted
EC2 Instances → Terminated
Application Load Balancer → Deleted
Target Group → Deleted
Launch Template → Deleted
Project Security Groups → Deleted

The cleanup ensured that the hands-on project did not leave unnecessary AWS resources running after testing.

Cleanup Order Used

1. Auto Scaling Group
↓
2. EC2 instances terminate
↓
3. Application Load Balancer
↓
4. Target Group
↓
5. Launch Template
↓
6. Security Groups
↓
7. Final resource verification

This completed the lifecycle of the project from **deployment** → **testing** → **failure simulation** → **recovery** → **cleanup**.